

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

Introduction

Instructor: Prof. Joshua San Miguel

Who am I?

Prof. Joshua San Miguel (Josh)

jsanmiguel@wisc.edu

Engineering Hall 3627

Office Hours: Wednesdays 12:30-1:30pm in EH3627

Education:

BASc in Eng Sci, University of Toronto, 2012

PhD, University of Toronto, 2017

Research Interests:

- Approximate and stochastic computing for low-power devices
- Unconventional processor architectures
- Caches, coherence protocols and memory systems
- Networks-on-chip for many-core processors
- Branch prediction, value prediction and prefetching
- Energy-harvesting IoT devices and intermittent computing



ECE/CS 757 Advanced Computer Architecture II

This course covers a range of topics in parallel computer architecture (and parallel programming).

Course Website: <https://sites.google.com/wisc.edu/ece757spring2025/>

Grades and Submission (Canvas): <https://canvas.wisc.edu/courses/434836>

Discussion Board: <https://piazza.com/wisc/spring2025/ececs757>

Lectures: MWF 11:00am-12:15pm in Brogden Psychology 121

Prerequisites: ECE/CS 552 Intro to Computer Architecture (or equivalent)

Course Grading

16%	Paper Reviews
5%	Quizzes
20%	Midterm Exam 1
20%	Midterm Exam 2
5%	Class Participation
34%	Project
➤ 14%	Topic Survey
➤ 18%	Proposal
➤ 18%	Status Report
➤ 25%	Lightning Talk Video
➤ 25%	Final Report

All work is due at 11:59pm, submitted through Canvas.

16% Paper Reviews

1. (Very) Short Summary (preferably as an analogy)
2. One Weakness
3. One Future Work
4. Sample Multiple-Choice Exam Question (and answer)

Page Limit:

Half a page

Helpful Reading:

A. J. Smith, *The Task of the Referee*, IEEE Computer, 1990.

5% Pop Quizzes

Each quiz covers any topics that have already been covered (either in class or assigned readings)

- Open book, open notes

20% + 20% Midterm Exams

Midterm 1:

Wed 2/26 in class

Midterm 2:

Wed 4/16 in class

- Formatted similar to past years' exams (to be posted on website)
- Sample questions in lectures (and some to be posted on website)
- Potential questions from your paper reviews
- No final exam

34% Project

- 14% Topic Survey
- 18% Proposal
- 18% Status Report
- 25% Lightning Talk Video
- 25% Final Report

34% Project

14% Topic Survey

Due: Friday 2/21

To be done individually:

Read three papers of potential project topic, and for each one (equal weighting):

1. Summarize paper
2. Discuss one idea for future work
3. Discuss another idea for future work

Page Limit:

One page

34% Project

18% Proposal

Due: Friday 3/07

To be done as a group:

Provide the following:

1. 250-word-limit abstract of future paper
2. Itemized list of project plan

Page Limit:

One page

34% Project

18% Status Report

Due: Friday 4/04

To be done as a group:

Provide the following:

1. 250-word-limit updated abstract of future paper
2. Itemized list of what you've done so far
3. Itemized list of what you plan to do next

Page Limit:

One page

34% Project

25% Lightning Talk Presentation Video

Due: Friday 5/09

To be done as a group:

Record a video that presents a high-level introduction of your project. It can be abstract and is only meant to serve as an elevator pitch to incite interest and give an idea of what problem you are addressing and how you go about solving it.

Time Limit:

90 seconds

34% Project

25% Final Report

Due: Friday 5/09

To be done as a group:

Write a two-column short paper for your project. You may reuse text from your topic survey, proposal and status report.

No need to submit project code.

Page Limit:

Four pages

5% Class Participation

Ask questions and discuss:

- In class
- At office hours
- With TA

Why Study Computer Architecture Now?

Mobile Computers
Graphics Processors Intermittent Computing
Internet-Of-Things Approximate Computing
Stochastic Computing Ultra-Low-Power Processors
In-Memory Computing Many-Core Processors Cloud Computing
Reconfigurable Architectures Quantum Computers
Energy-Harvesting Processors Near-Threshold Computing
Warehouse-Scale Computers Biodegradable Processors
Neuromorphic Processors

Why Study Computer Architecture Now?

Mobile Computers
Graphics Processors Intermittent Computing
Internet-Of-Things Approximate Computing
Stochastic Computing Ultra-Low-Power Processors
In-Memory Computing **How did we get here?** Cloud Computing
Reconfigurable Architectures Quantum Computers
Energy-Harvesting Processors Near-Threshold Computing
Warehouse-Scale Computers Biodegradable Processors
Neuromorphic Processors

Dennard Scaling

Robert H. Dennard et al., *Design of Ion-Implanted MOSFET's with Very Small Dimensions*, IEEE Journal of Solid-State Circuits, 1974.

Transistor scaling:

$$\text{area } A \propto 1/X^2$$

$$\text{voltage } V \propto 1/X$$

$$\text{capacitance } C \propto 1/X$$

$$\text{switching energy } E \propto 1/X^3$$

$$\text{delay } T \propto 1/X$$



Dennard Scaling

Robert H. Dennard et al., *Design of Ion-Implanted MOSFET's with Very Small Dimensions*, IEEE Journal of Solid-State Circuits, 1974.

Transistor scaling:

$$\begin{aligned} \text{area } A &\propto 1/X^2 \\ \text{voltage } V &\propto 1/X \\ \text{capacitance } C &\propto 1/X \\ \text{switching energy } E &\propto 1/X^3 \\ \text{delay } T &\propto 1/X \end{aligned}$$

Processor dynamic power:

$$P_D \approx \underset{\substack{\text{number of transistors}}}{n} \cdot (C \cdot V^2) \cdot (\underset{\substack{\text{activity factor}}}{\alpha} \cdot \underset{\substack{\text{frequency}}}{f})$$



Dennard Scaling

Robert H. Dennard et al., *Design of Ion-Implanted MOSFET's with Very Small Dimensions*, IEEE Journal of Solid-State Circuits, 1974.

Transistor scaling:

$$\begin{aligned} \text{area } A &\propto 1/X^2 \\ \text{voltage } V &\propto 1/X \\ \text{capacitance } C &\propto 1/X \\ \text{switching energy } E &\propto 1/X^3 \\ \text{delay } T &\propto 1/X \end{aligned}$$

Processor dynamic power:

$$P_D \approx n \cdot \underbrace{(C \cdot V^2)}_{1/X^3} \cdot (\alpha \cdot f)$$

Dennard Scaling

Robert H. Dennard et al., *Design of Ion-Implanted MOSFET's with Very Small Dimensions*, IEEE Journal of Solid-State Circuits, 1974.

Transistor scaling:

$$\begin{aligned} \text{area } A &\propto 1/X^2 \\ \text{voltage } V &\propto 1/X \\ \text{capacitance } C &\propto 1/X \\ \text{switching energy } E &\propto 1/X^3 \\ \text{delay } T &\propto 1/X \end{aligned}$$

Processor dynamic power:

$$P_D \approx n \cdot \underbrace{(C \cdot V^2)}_{1/X^3} \cdot \underbrace{(\alpha \cdot f)}_X$$

Dennard Scaling

Robert H. Dennard et al., *Design of Ion-Implanted MOSFET's with Very Small Dimensions*, IEEE Journal of Solid-State Circuits, 1974.

Transistor scaling:

$$\text{area } A \propto 1/X^2$$

$$\text{voltage } V \propto 1/X$$

$$\text{capacitance } C \propto 1/X$$

$$\text{switching energy } E \propto 1/X^3$$

$$\text{delay } T \propto 1/X$$

Processor dynamic power:

$$P_D \approx \underbrace{n}_{X^2} \cdot \underbrace{(C \cdot V^2)}_{1/X^3} \cdot \underbrace{(\alpha \cdot f)}_X$$



Dennard Scaling

Robert H. Dennard et al., *Design of Ion-Implanted MOSFET's with Very Small Dimensions*, IEEE Journal of Solid-State Circuits, 1974.

Transistor scaling:

$$\text{area } A \propto 1/X^2$$

$$\text{voltage } V \propto 1/X$$

$$\text{capacitance } C \propto 1/X$$

$$\text{switching energy } E \propto 1/X^3$$

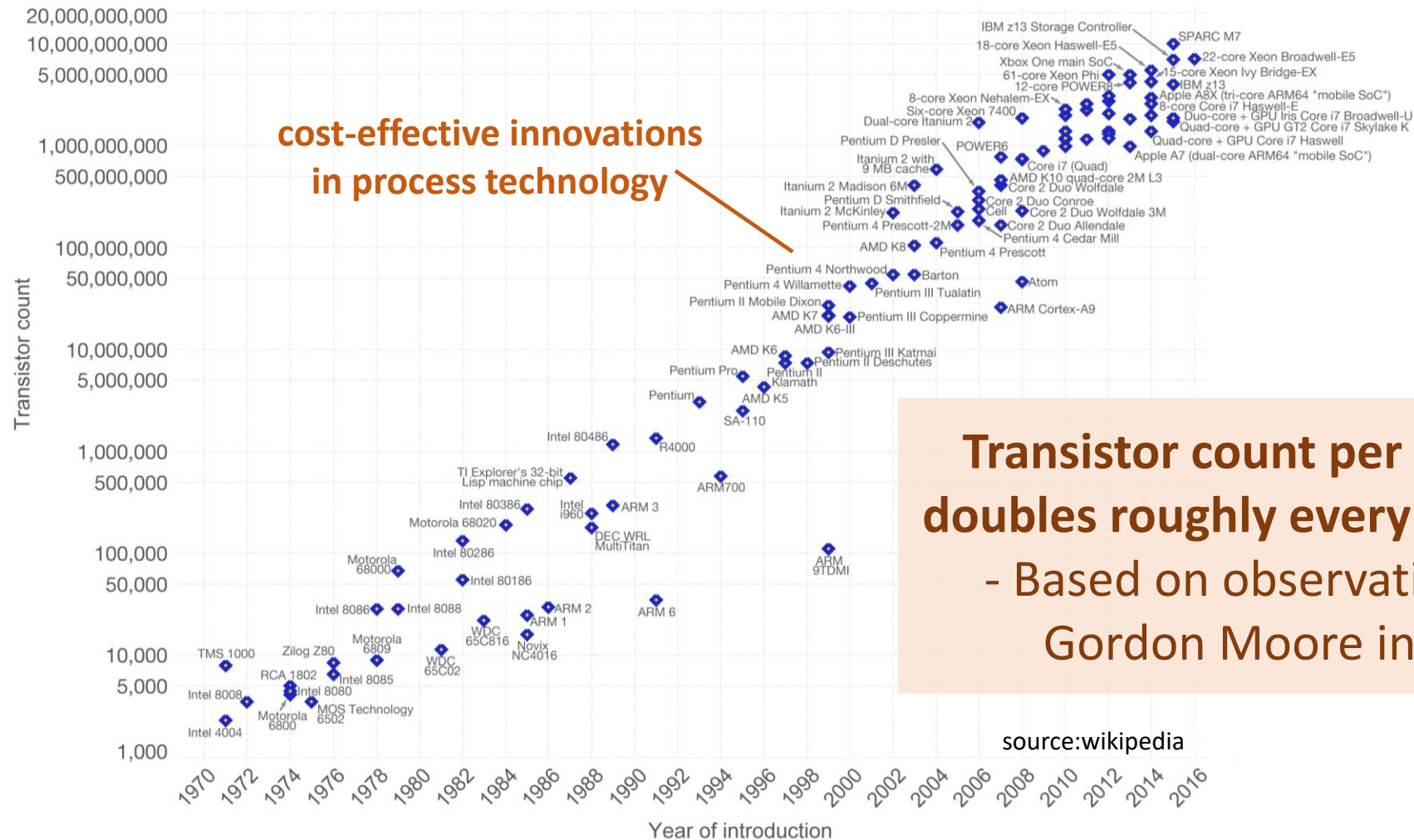
$$\text{delay } T \propto 1/X$$

Processor dynamic power:

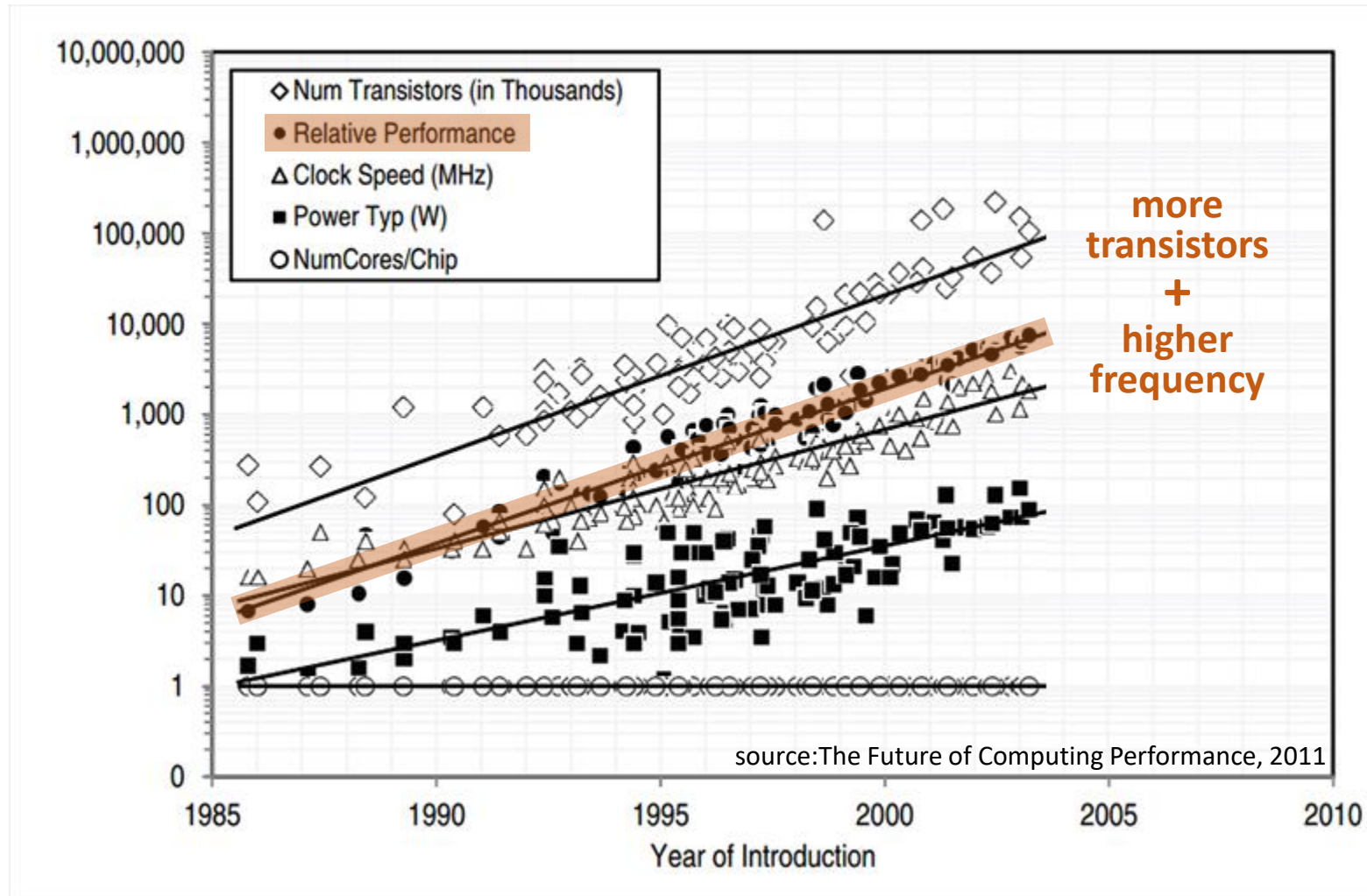
$$P_D \approx \underbrace{n}_{\substack{\text{more} \\ \text{transistors} \\ X^2}} \cdot \underbrace{(C \cdot V^2)}_{1/X^3} \cdot \underbrace{(\alpha \cdot f)}_{\substack{\text{higher} \\ \text{frequency} \\ X}}$$

similar dynamic power!

Moore's Law



Moore's Law + Dennard Scaling



...But Leakage Power

Processor static (leakage) power:

$$P_S \approx n \cdot V \cdot e^{-V_{th}}$$

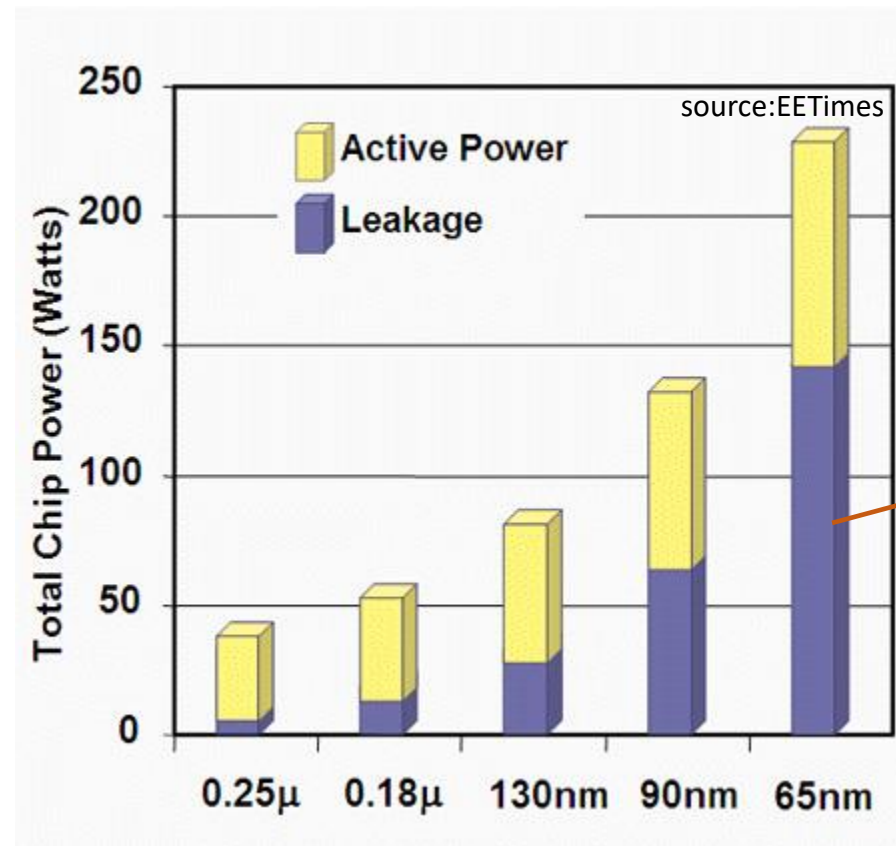
threshold voltage

...But Leakage Power

Processor static (leakage) power:

$$P_S \approx n \cdot V \cdot e^{-V_{th}}$$

threshold voltage



useless power consumption exponentially increasing!

End of Dennard Scaling

Robert H. Dennard et al., *Design of Ion-Implanted MOSFET's with Very Small Dimensions*, IEEE Journal of Solid-State Circuits, 1974.

Transistor scaling:

$$\text{area } A \propto 1/X^2$$

$$\text{voltage } V \propto 1/X$$

$$\text{capacitance } C \propto 1/X$$

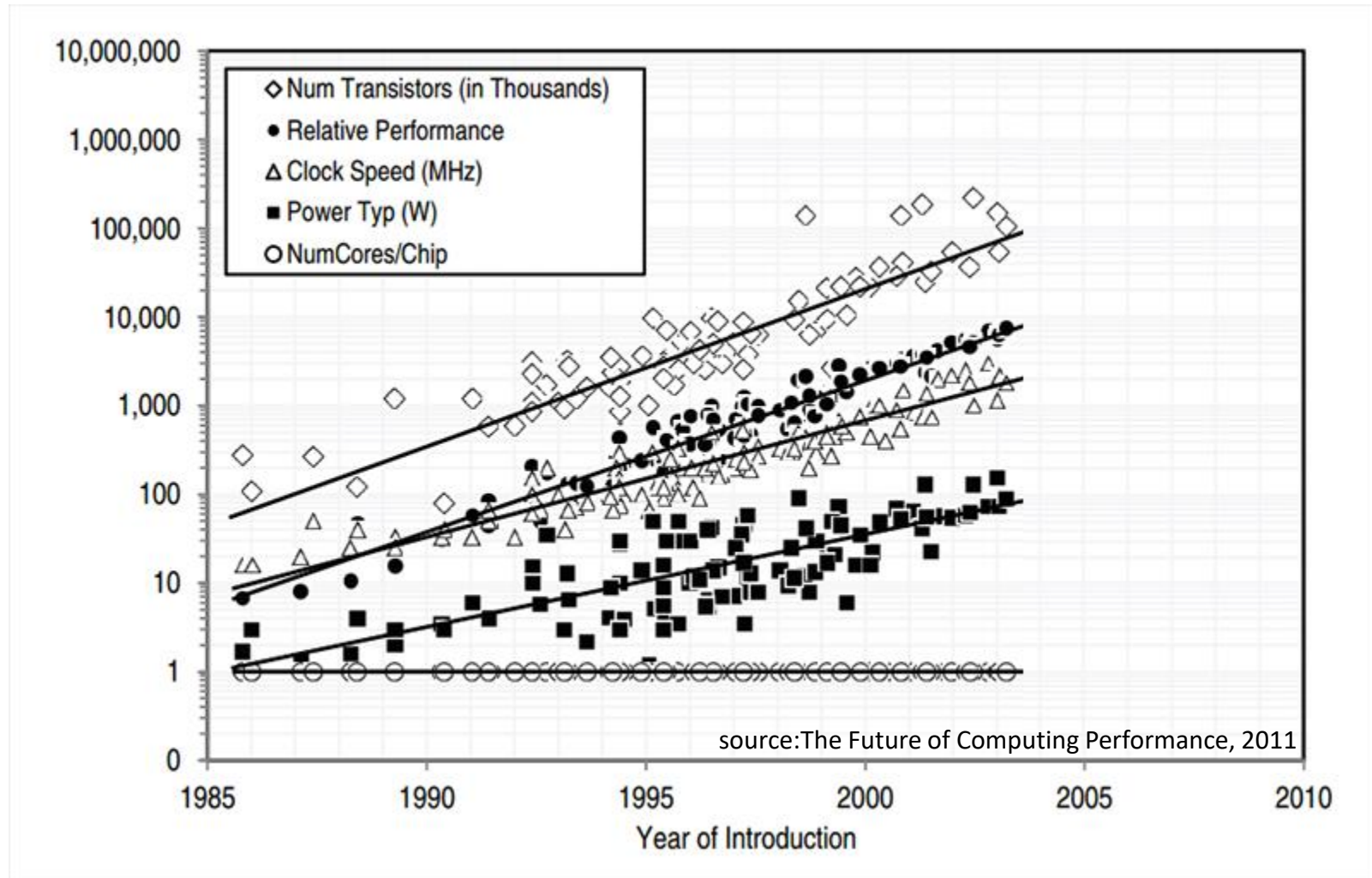
$$\text{switching energy } E \propto 1/X^3$$

$$\text{delay } T \propto 1/X$$

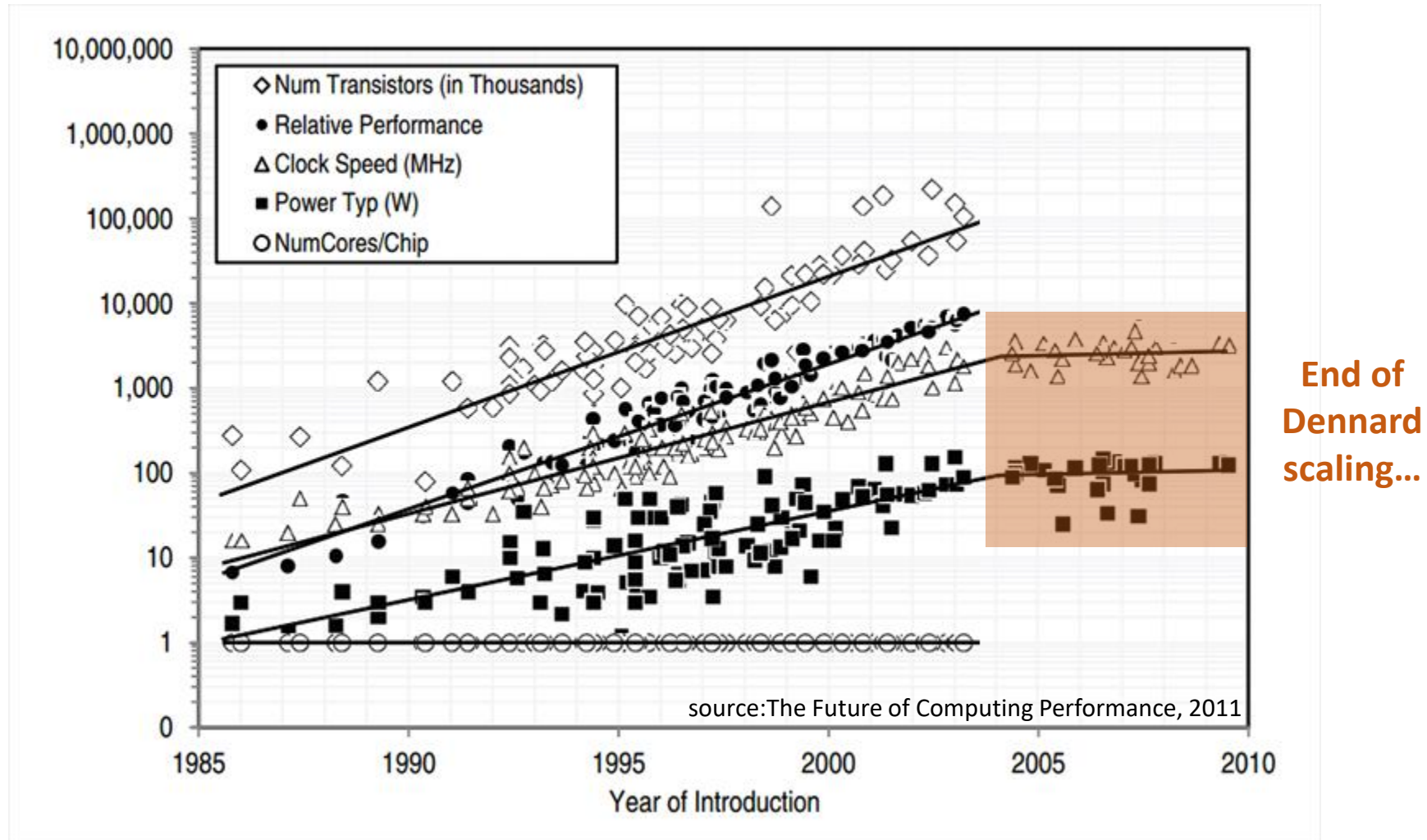
X quickly diminishing due to leakage!



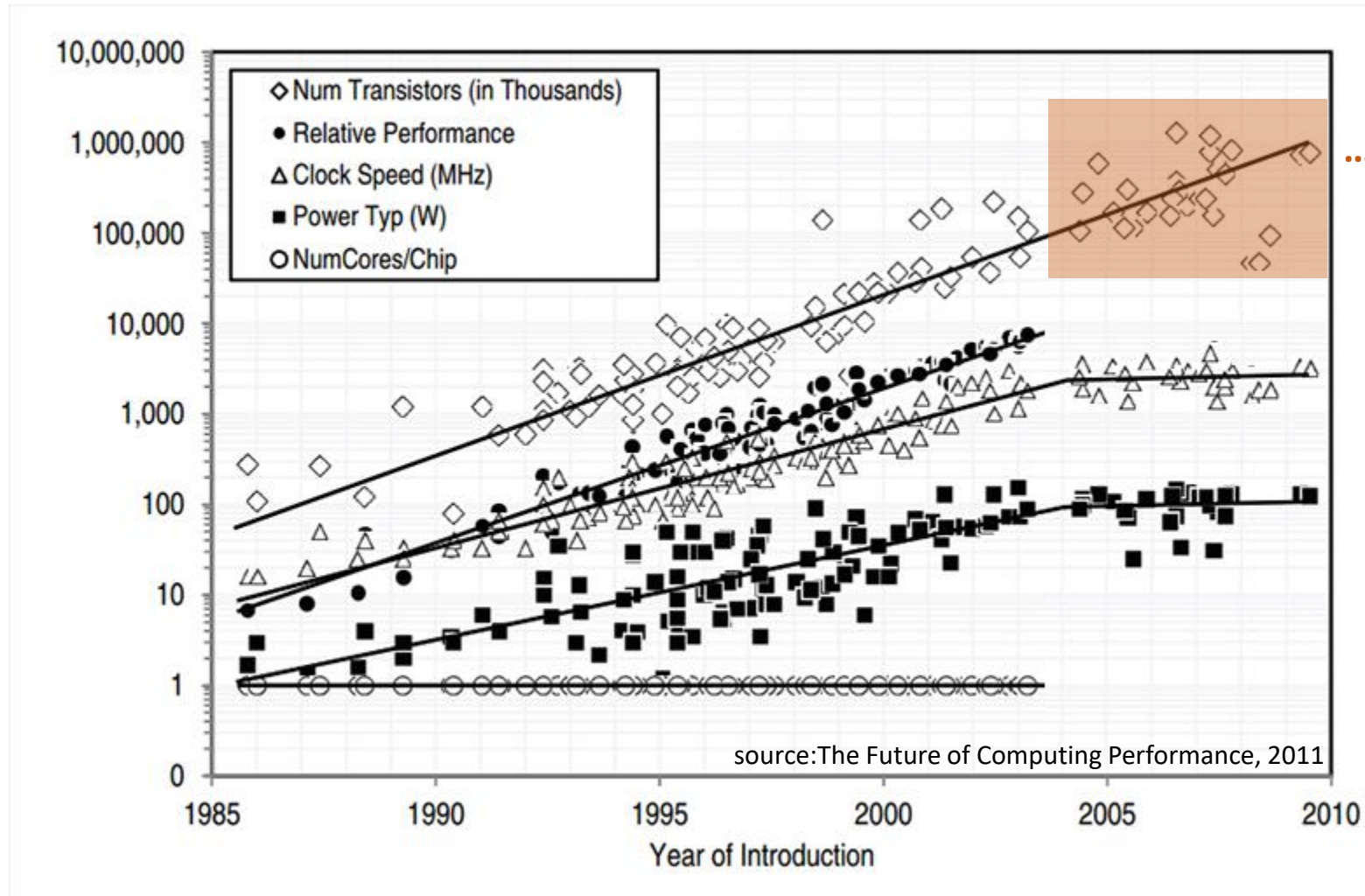
End of Dennard Scaling



End of Dennard Scaling



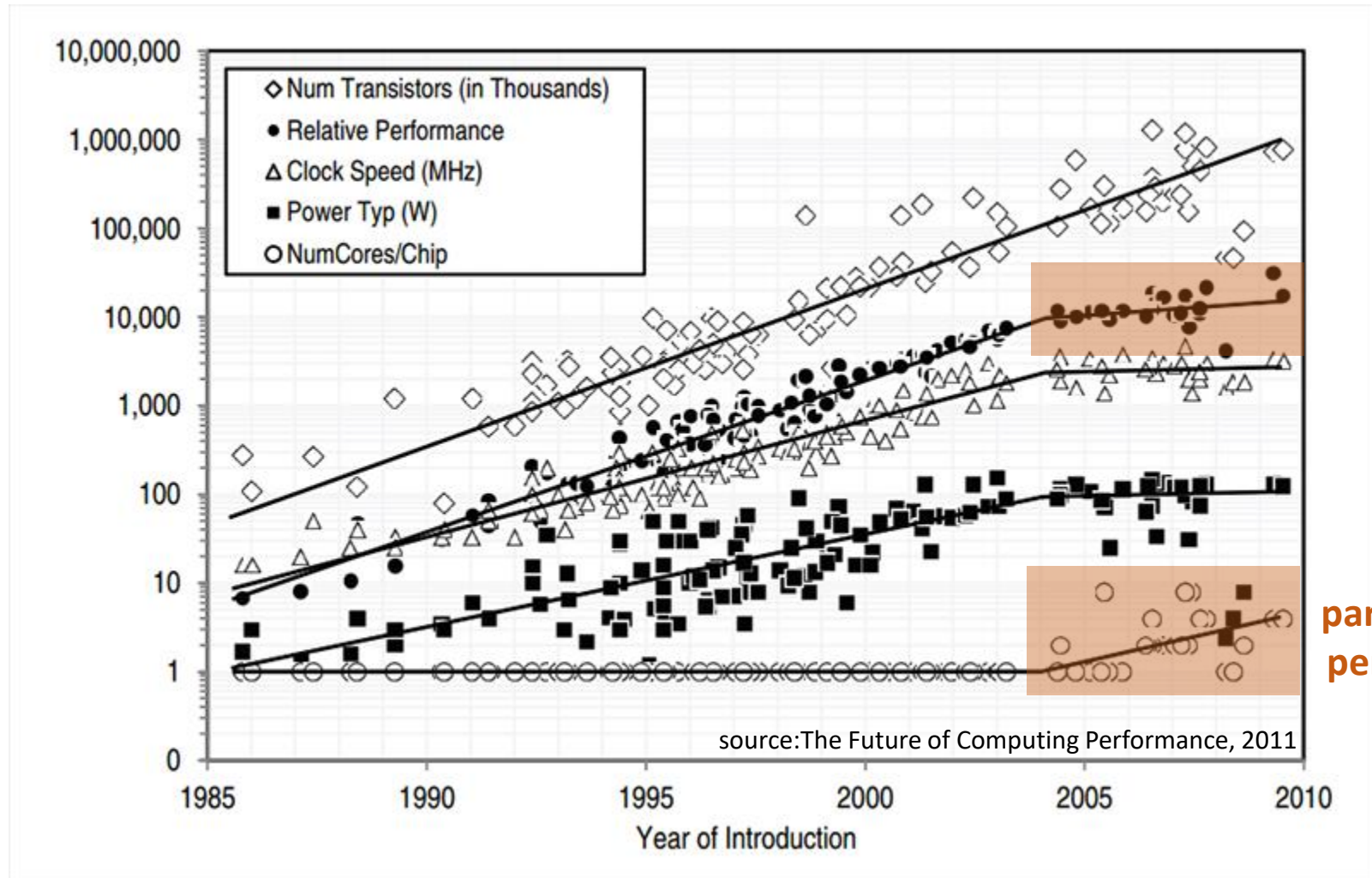
End of Dennard Scaling



...but Moore's Law persists!



Moore's Law – Dennard Scaling = Parallelization

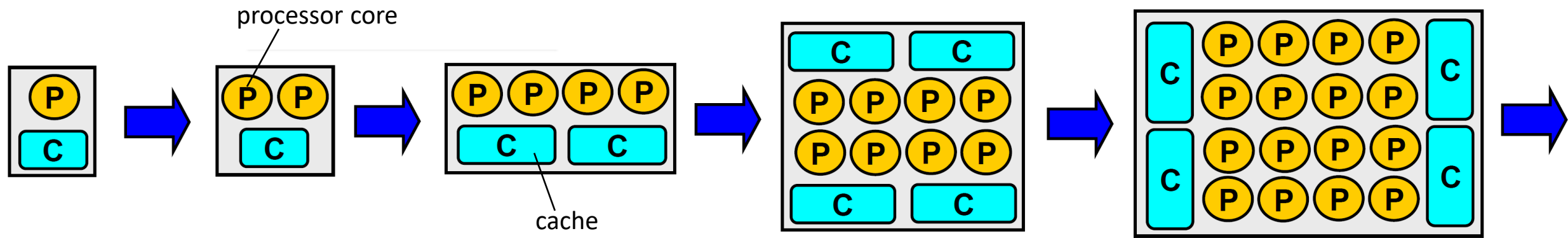


parallelize for
performance



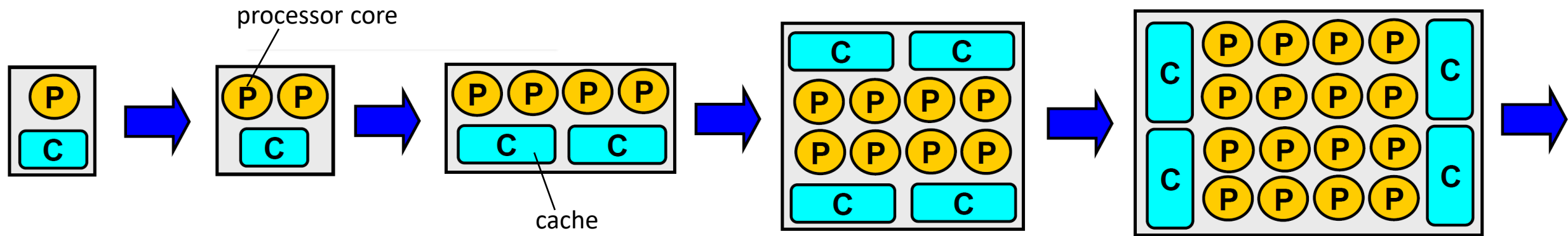
Moore's Law – Dennard Scaling = Parallelization

“The core is the logic gate of the 21st century”



Moore's Law – Dennard Scaling = Parallelization

“The core is the logic gate of the 21st century”



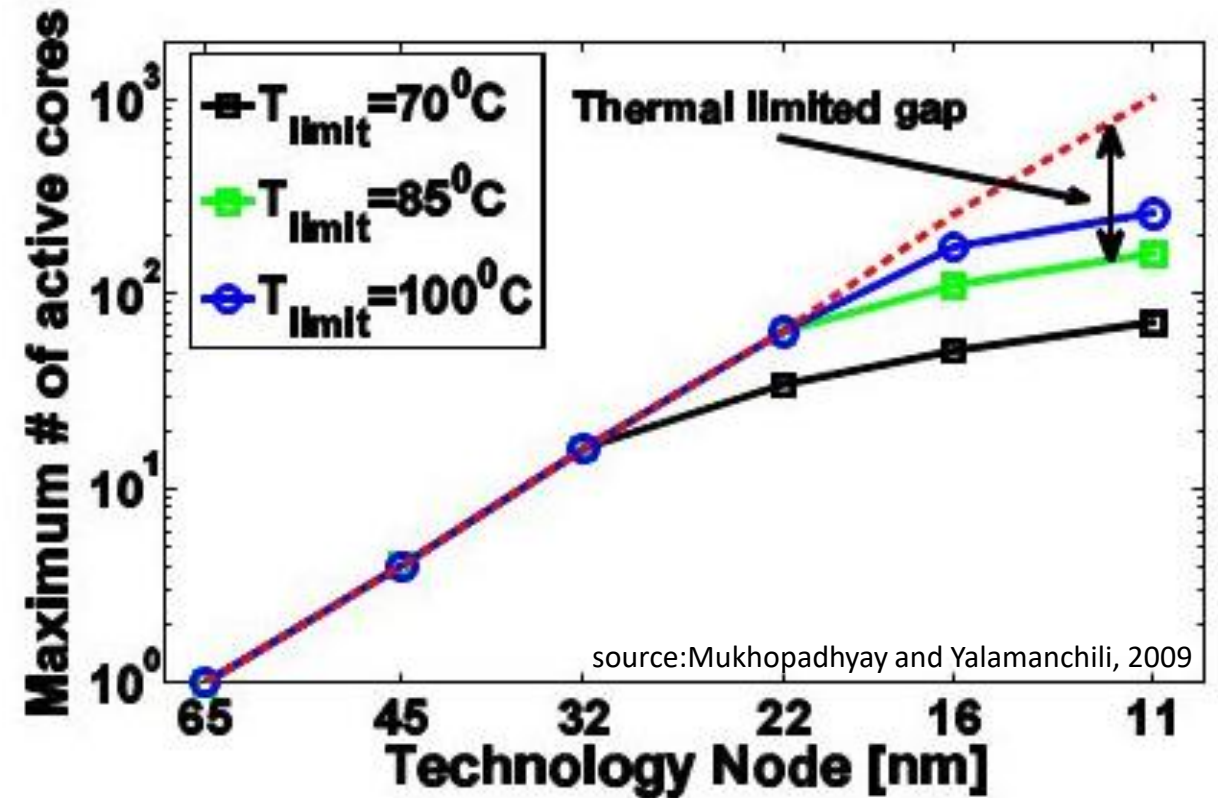
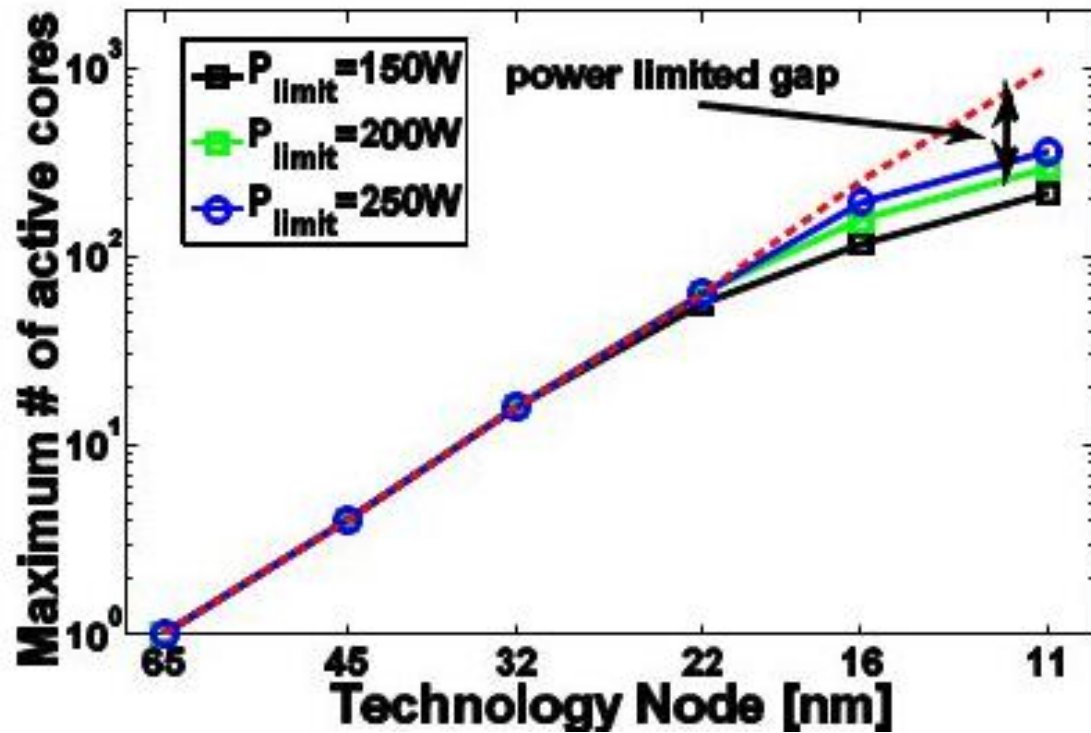
+ Tolerates cross-chip wire latency
(localized wires)

+ Lower idle power
(turn off unused/waiting cores)

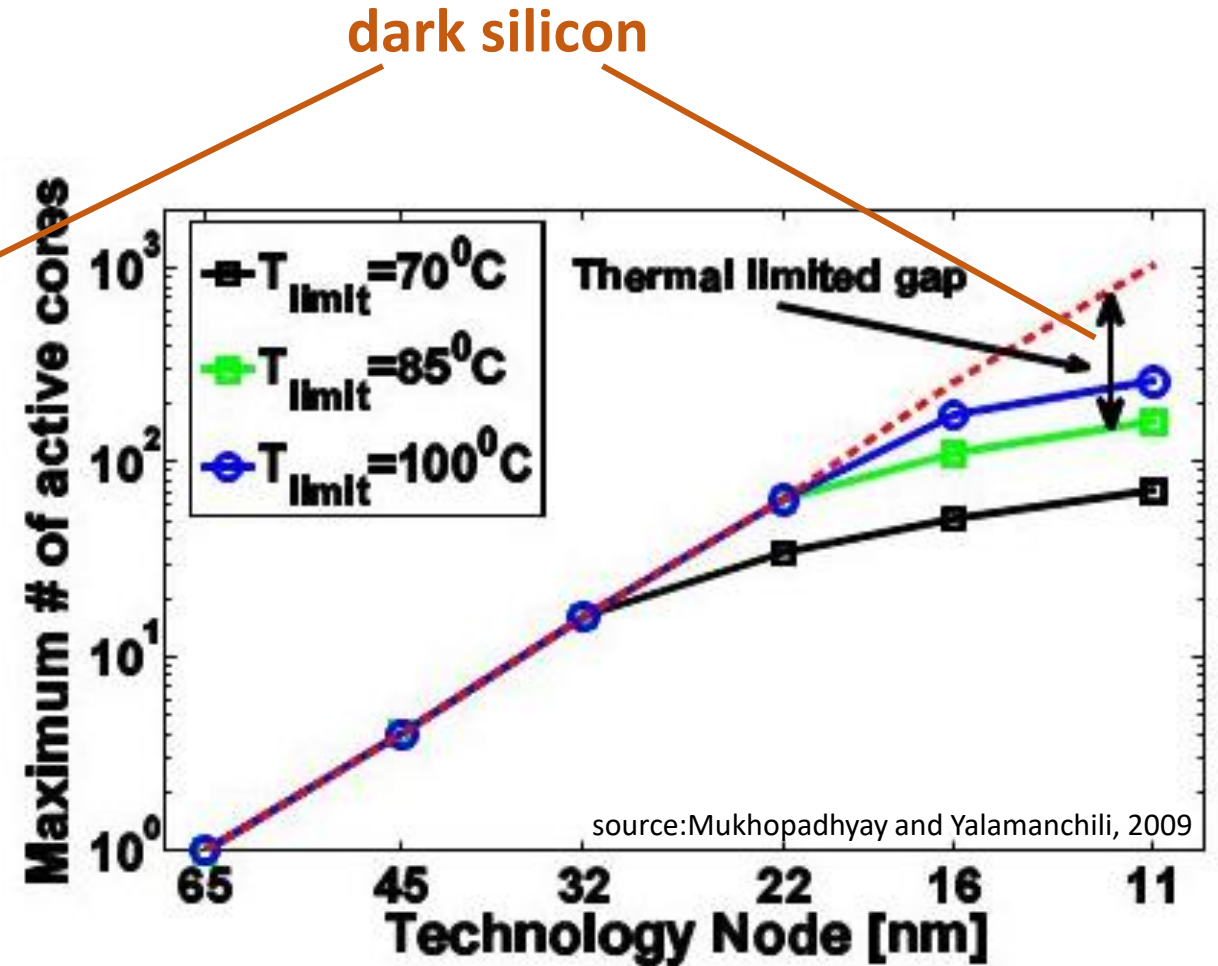
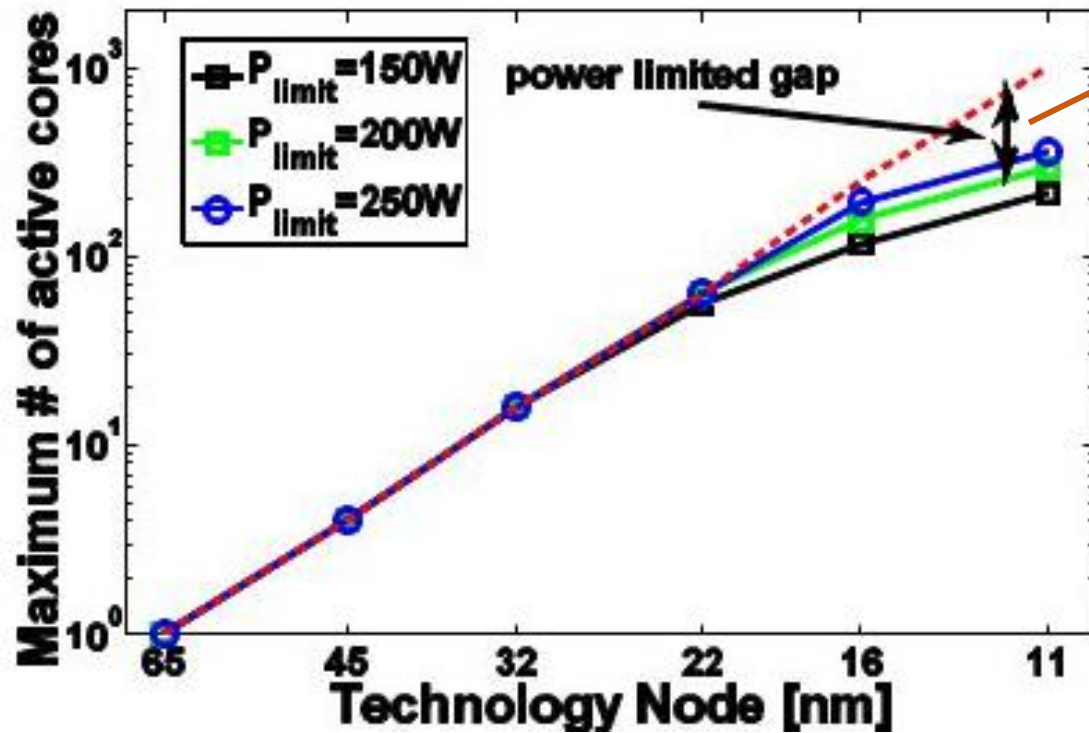
+ Lower development cost
(stamp out processor cores)

+ Tolerate defects
(disable any faulty cores)

...But Increasing Gap in Power Density Scaling

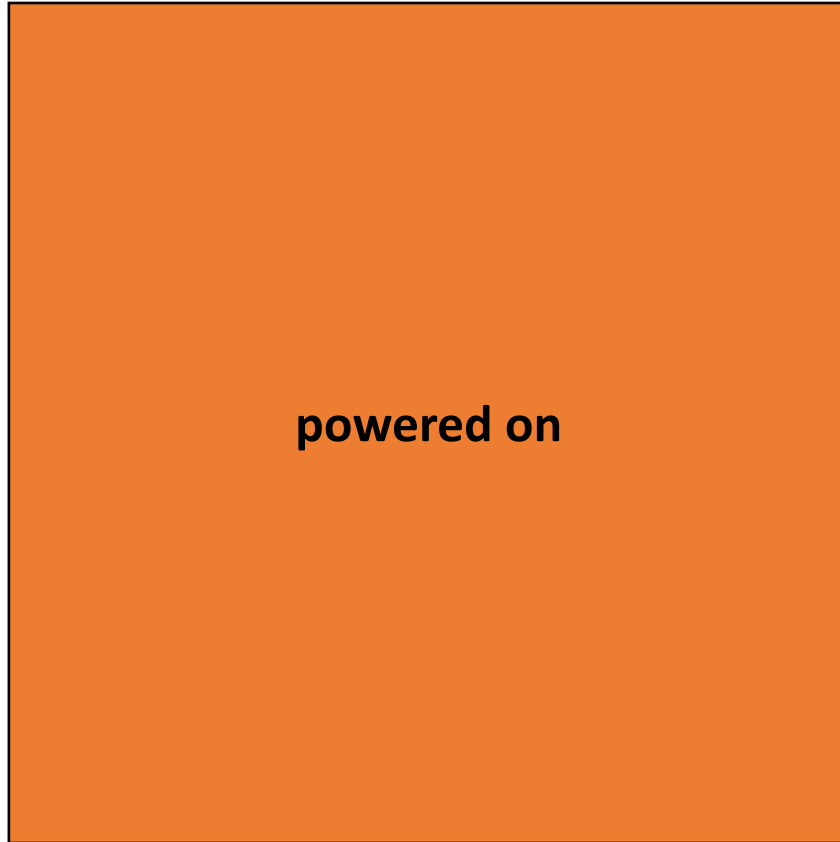


...But Increasing Gap in Power Density Scaling

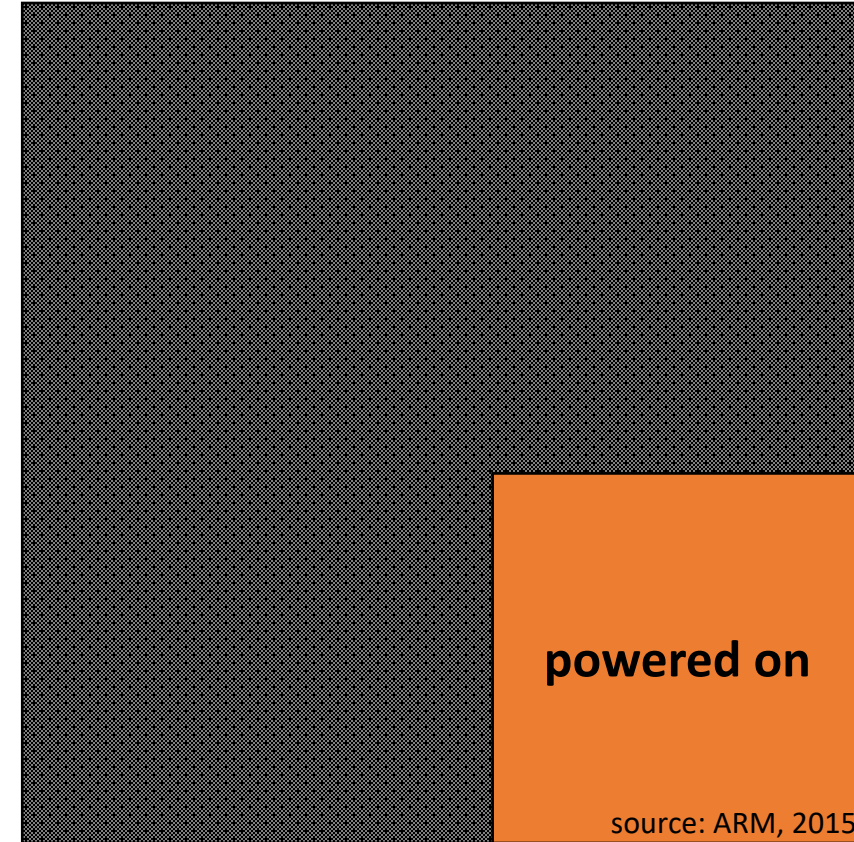


Dark Silicon

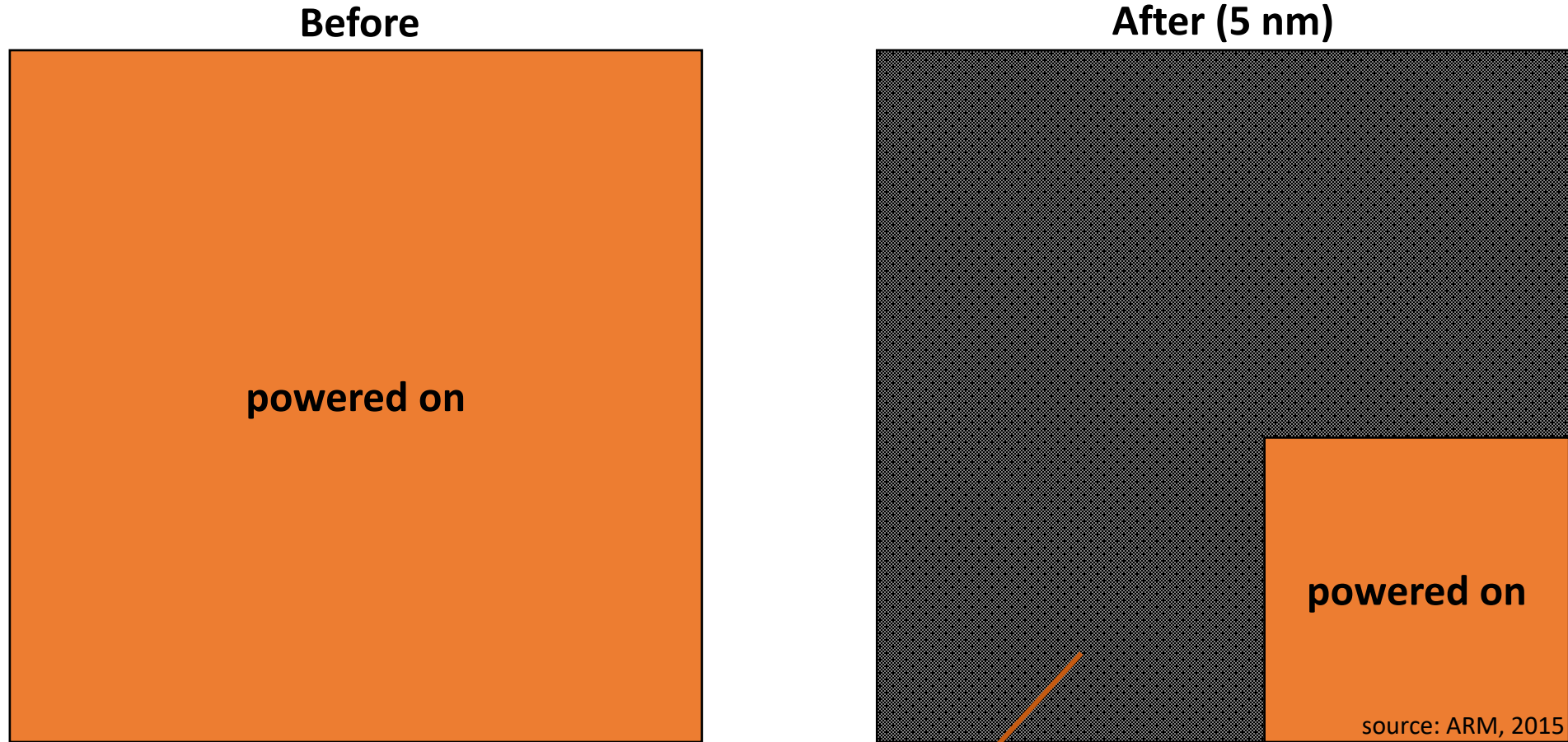
Before



After (5 nm)



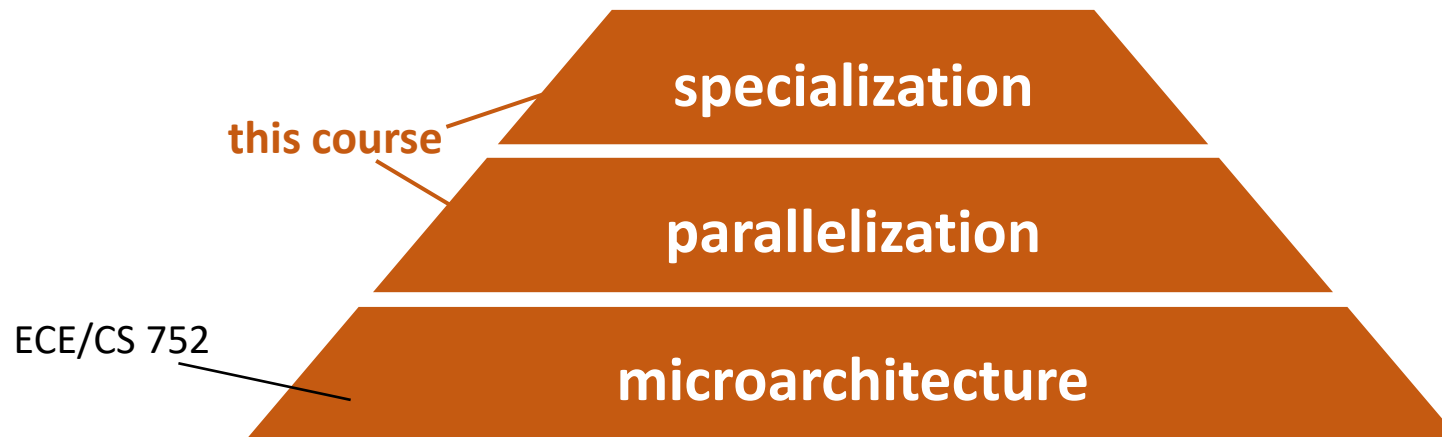
Moore's Law + Dark Silicon = Specialization



fill with specialized (heterogeneous) cores

...And Computer Architecture Now

Mobile Computers
Graphics Processors Intermittent Computing
Internet-Of-Things Approximate Computing
Stochastic Computing Ultra-Low-Power Processors
In-Memory Computing Many-Core Processors Cloud Computing
Reconfigurable Architectures Quantum Computers
Energy-Harvesting Processors Near-Threshold Computing
Warehouse-Scale Computers Biodegradable Processors
Neuromorphic Processors



Parallel Architectures are Here to Stay

“The core is the logic gate of the 21st century”

Intel Xeon Platinum
(56 hardware threads)



NVIDIA Ampere
(10752 hardware threads)



IBM Blue Gene/Q Sequoia
(1.6 million cores)



Research Takeaways

Key research goals of this course:

1. Think **critically**
 - Everything has trade-offs and weaknesses
2. Think **high-level**
 - Focus on insights/ideas not numbers
3. Think in **analogies**
 - Draw parallels and contrasts
4. Think **iteratively**
 - Work towards one little win at a time
5. Think in **fundamentals**
 - Communicate thoughts concisely

Reading Research Papers

Master the art of skimming:

1. Learn where to quickly find the “point”

- Remember that the authors are just trying to tell a story.
- Somehow you can tell when something bad is about to happen in a movie.

2. Learn to read a paper as if you wrote it yourself

- Remember that the authors are probably students just like you.
- Somehow you know every flaw in your own work like the back of your hand.

3. Learn to always ask “how is this different from X?”

- Remember that the authors are just trying to sell you something.
- Somehow you’re all experts at what not to buy (and what not to click) online.

Technical Takeaways

Key design trade-offs of parallel architectures:

1. Architecture **generality** vs. **efficiency**
2. Programming **productivity** vs. **performance**
3. Cache **locality** vs. **capacity**
4. Memory **safety** vs. **latency**
5. Interconnect **scalability** vs. **complexity**
6. Accelerator **programmability** vs. **overhead**

Technical Takeaways

1. Architecture **generality** vs. **efficiency**:

Maximize average efficiency for a broad range of applications?

OR

Favor peak efficiency for a specific subset of applications?

Technical Takeaways

2. Programming **productivity** vs. **performance**:

Minimize programmer burden by simplifying the parallelization interface?

OR

Maximize potential performance by exposing comprehensive parameters?

```
parallel task {  
    lock(ALL);  
    if (A == ...) {  
        // do something with X  
        ...  
        // do something with Y  
        A = ...;  
    }  
    unlock(ALL);  
}
```

VS

```
parallel task {  
    lock(A);  
    if (A == ...) {  
        unlock(A);  
        lock(X);  
        // do something with X  
        unlock(X);  
        ...  
        lock(Y);  
        // do something with Y  
        unlock(Y);  
        lock(A);  
        A = ...;  
    }  
    unlock(A);  
}
```

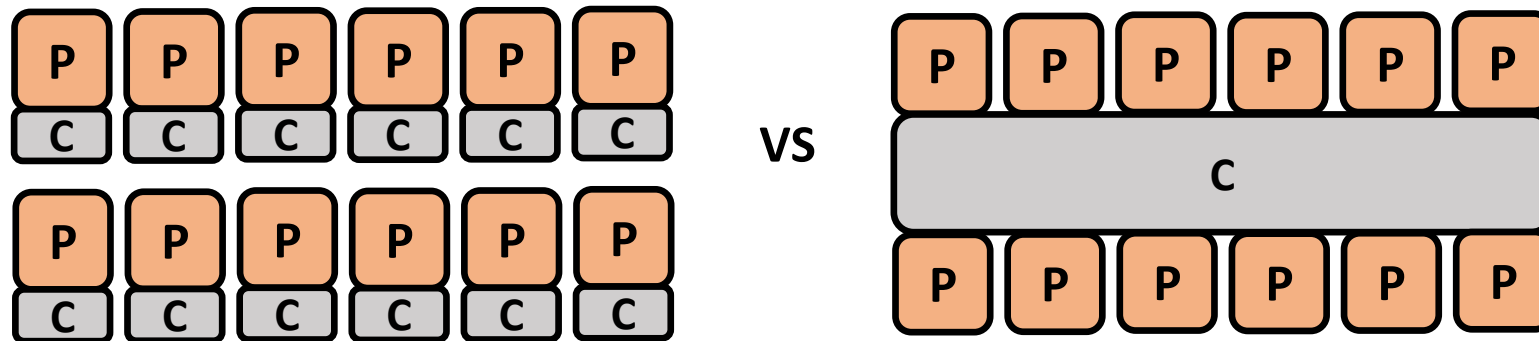
Technical Takeaways

3. Cache **locality** vs. **capacity**:

Keep data as close as possible to the processor cores that need them?

OR

Maximize amount of data stored in on-chip caches?



Technical Takeaways

4. Memory **safety** vs. **latency**:

Lower risk of programming bugs by simplifying memory semantics?

OR

Minimize amount of time spent waiting to access data from memory?

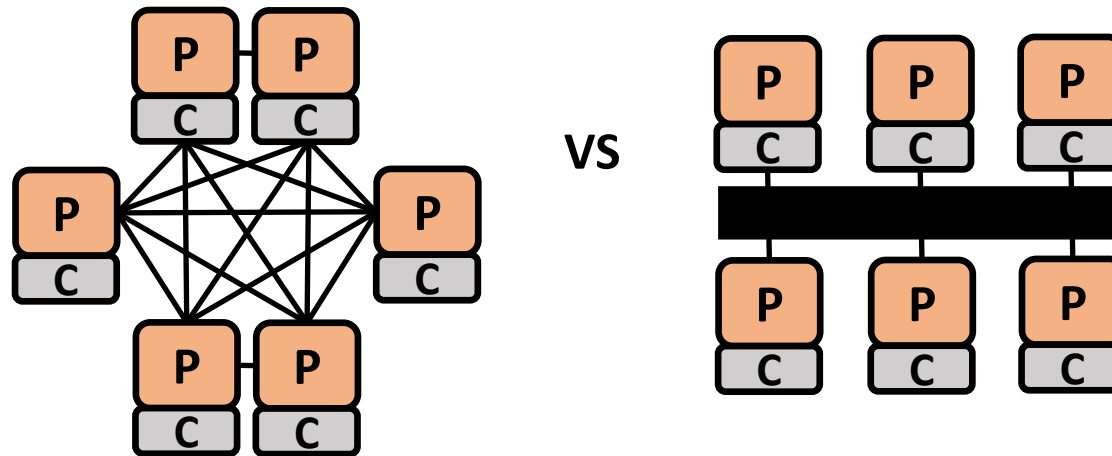
Technical Takeaways

5. Interconnect **scalability** vs. **complexity**:

Maximize network throughput when more and more cores are added?

OR

Minimize network footprint by simplifying routing and wiring?



Technical Takeaways

6. Accelerator **programmability** vs. **overhead**:

Enable support for more kinds of computations to use accelerator?

OR

Minimize overhead of configuring and invoking accelerator?

It Depends

“It depends” is almost always the right answer

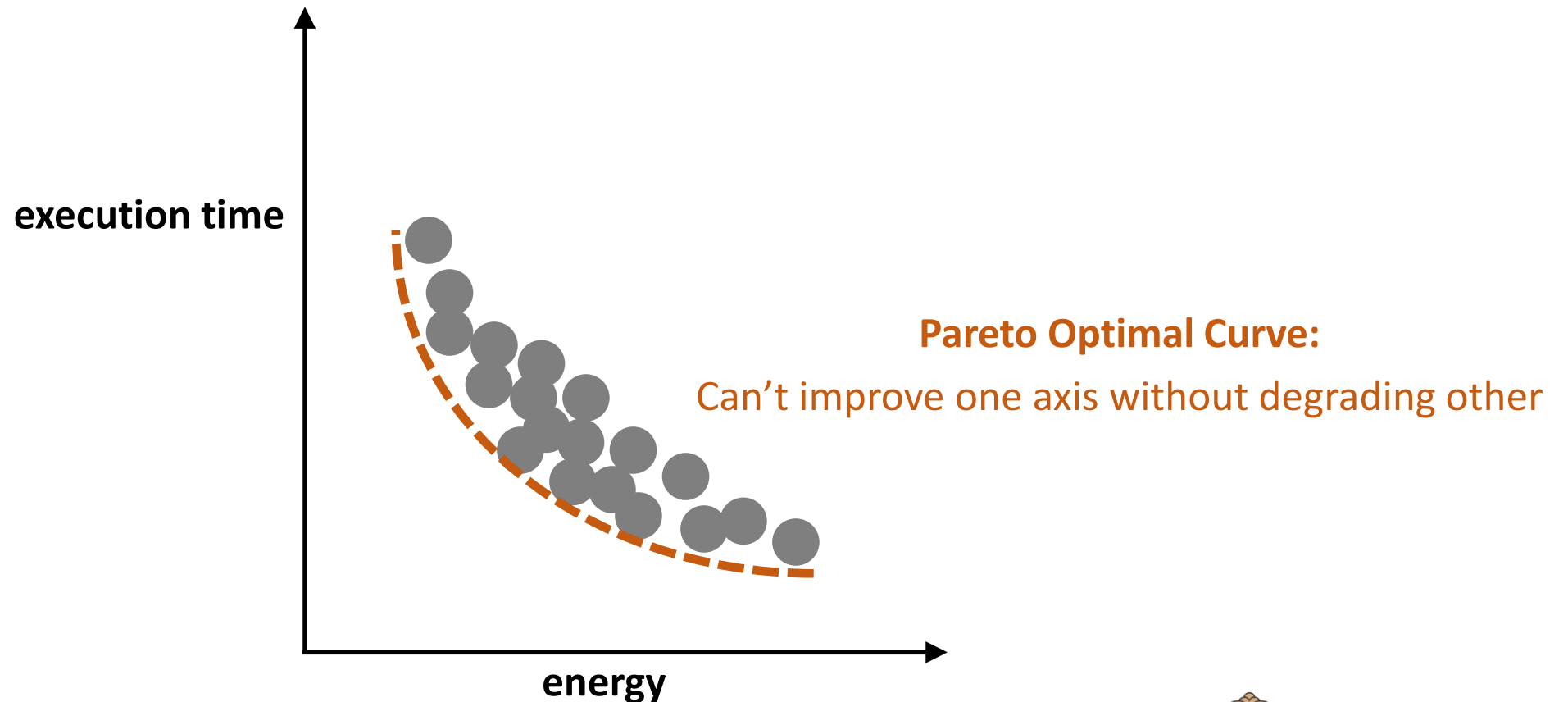
...But it's an insufficient one.

Your goal in designing and evaluating an architecture is to discover and formulate its trade-offs:

- Under what *circumstances* (i.e., app, device) does it perform better or worse?
- What is its *pathological worst-case* scenario? Pathological best-case?
- Where is the *inflection point* at which prior work becomes inferior or superior?
- What are its *optimal parameters* under varying use cases?

Design Trade-Offs – Pareto Optimal

There is no such thing as a one-size-fits-all design



Computer Architecture in Perspective

Mark D. Hill et al., *21st Century Computer Architecture*, Computing Community Consortium Whitepaper, 2012.

- Computation in isolation
- Single-chip systems
- Performance first
- Software-invisible design
- Tried and tested technologies

Computer Architecture in Perspective

Mark D. Hill et al., *21st Century Computer Architecture*, Computing Community Consortium Whitepaper, 2012.

- ~~Computation in isolation~~ **Computation in context (sensors, cloud)**
- ~~Single-chip systems~~ **Architecture as infrastructure**
- ~~Performance first~~ **Energy first (plus security, dependability)**
- ~~Software-invisible design~~ **Cross-layer design (specialization)**
- ~~Tried and tested technologies~~ **Non-volatile, 3D, photonics, etc.**

Computer Architecture in Perspective

Application Pull

Technology Push

Homework 0

Fill out questionnaire on Canvas

Due: Friday 1/24